



E4 Educator v2.0

T05

OLED display

Tier 1 · Tutorial 5 of 7

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- How the SSD1306 OLED display works and why it is the ideal entry-level display
- The 128×64 pixel grid and how to think about layout within its constraints
- How to initialise the display using the Adafruit SSD1306 library
- How to draw text, lines, rectangles, and circles
- How to build a clean multi-zone display layout
- How to avoid display flicker using the display buffer correctly
- How to build a complete sensor-to-screen project with the AHT20
- How to create a simple multi-page display driven by button events

You will need:

- E4 Educator v2.0 board with ESP32, AHT20, and OLED fitted
- T01 through T04 completed — Button.h available
- USB-C cable (data-capable)

1 The SSD1306 OLED display

The 0.96" SSD1306 OLED is a monochrome display with 128 pixels wide by 64 pixels tall. OLED stands for Organic Light Emitting Diode — each pixel generates its own light, which means there is no backlight, contrast is excellent, and blacks are truly black.

It connects to the E4 via I2C — the same SDA and SCL lines you used for the AHT20 in T04. This is why the OLED is the ideal entry-level display: two wires, no configuration file, and it shares the bus already set up in your project.

1.1 Key characteristics

Parameter	Detail
Resolution	128 × 64 pixels — monochrome (white pixels on black)
Interface	I2C — SDA GPIO 21, SCL GPIO 22. No additional wiring beyond the OLED header.
I2C address	0x3C (default). Some modules use 0x3D via solder bridge.
Supply voltage	3.3V via OLED header — 3V3 devices only
Display area	Approximately 21mm × 11mm physical display area
Pixel addressing	X: 0 (left) to 127 (right). Y: 0 (top) to 63 (bottom).

1.2 Thinking in 128×64

128×64 feels limiting at first. It is actually a useful constraint — it forces you to think carefully about what information matters and how to present it clearly. These are skills that transfer directly to larger display projects.

A practical mental model for the 128×64 grid:

Font size	Characters per row	Rows available
Size 1 (6×8px)	21 characters	8 rows — good for dense data displays
Size 2 (12×16px)	10 characters	4 rows — good balance of readability and data density
Size 3 (18×24px)	7 characters	2 rows — large values, status displays

★ Key point

The 128×64 constraint is a feature, not a limitation. It teaches you to prioritise information — a skill every embedded UI developer needs. When you move to the TFT in Tier 2, the larger canvas will feel genuinely liberating rather than taken for granted.

2 Library setup and first display

2.1 Add the library

Add the SSD1306 library to your platformio.ini lib_deps:

```
[env:e4_educator_v2]
; ... existing settings ...
lib_deps =
    adafruit/Adafruit SSD1306
    adafruit/Adafruit GFX Library
    adafruit/Adafruit BusIO
    adafruit/Adafruit AHTX0
```

2.2 Hello OLED

Create a new project called E4_T05_OLED and enter this into main.cpp:

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define OLED_WIDTH 128
#define OLED_HEIGHT 64
#define OLED_RESET -1 // No reset pin on E4 OLED header
#define OLED_ADDR 0x3C

Adafruit_SSD1306 oled(OLED_WIDTH, OLED_HEIGHT, &Wire, OLED_RESET);

void setup() {
    Serial.begin(115200);
    Serial.printf("T05 - Hello OLED FW: %s\n", FW_VERSION);

    Wire.begin(I2C_SDA, I2C_SCL);

    if (!oled.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
        Serial.println("ERROR: SSD1306 not found. Check OLED header.");
        while (true) { delay(100); }
    }

    // Clear the buffer — always do this first
    oled.clearDisplay();

    // Set text properties
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(2);
    oled.setCursor(10, 10);
    oled.println("Hello");

    oled.setTextSize(1);
    oled.setCursor(10, 35);
```

```
oled.println("E4 Educator v2.0");

oled.setCursor(10, 50);
oled.printf("FW: %s", FW_VERSION);

// Push buffer to display — nothing shows until this is called
oled.display();
Serial.println("OLED ready.");
}

void loop() {
  // Static display — nothing to update
}
```

The display buffer pattern

The SSD1306 library draws into a memory buffer first. Nothing appears on the display until you call `oled.display()`. This is important — always build your complete screen in the buffer, then push it all at once. This prevents flicker and gives smooth updates.

3 Drawing primitives

Beyond text, the Adafruit GFX library provides a set of drawing functions for lines, rectangles, circles, and pixels. These are the building blocks of any custom display layout.

```
// — Pixels —————
oled.drawPixel(x, y, SSD1306_WHITE);

// — Lines —————
oled.drawLine(x0, y0, x1, y1, SSD1306_WHITE);
oled.drawFastHLine(x, y, width, SSD1306_WHITE); // Faster horizontal line
oled.drawFastVLine(x, y, height, SSD1306_WHITE); // Faster vertical line

// — Rectangles —————
oled.drawRect(x, y, width, height, SSD1306_WHITE); // Outline
oled.fillRect(x, y, width, height, SSD1306_WHITE); // Filled
oled.fillRect(x, y, width, height, SSD1306_BLACK); // Erase area

// — Rounded rectangles —————
oled.drawRoundRect(x, y, width, height, radius, SSD1306_WHITE);
oled.fillRoundRect(x, y, width, height, radius, SSD1306_WHITE);

// — Circles —————
oled.drawCircle(centreX, centreY, radius, SSD1306_WHITE);
oled.fillCircle(centreX, centreY, radius, SSD1306_WHITE);

// — Text —————
oled.setTextSize(1); // 1=small, 2=medium, 3=large
oled.setTextColor(SSD1306_WHITE);
oled.setTextWrap(false); // Prevent text wrapping at edges
oled.setCursor(x, y);
oled.print("text");
oled.printf("Temp: %.1f", tempC);

// — Always push to display after drawing —————
oled.display();
```

3.1 Erasing areas

To update part of the display without clearing everything, use `fillRect()` with `SSD1306_BLACK` to erase just the area you want to update, then redraw it:

```
// Erase just the value area, then redraw
oled.fillRect(0, 24, 128, 16, SSD1306_BLACK); // Erase row
oled.setCursor(0, 24);
oled.setTextSize(2);
oled.printf("%.1f C", temperature);
oled.display();
```

★ Key point

Never call `oled.clearDisplay()` every `loop()` pass unless you are redrawing the entire screen. Clearing and redrawing everything causes visible flicker. Instead, erase only the zones that change using `fillRect()` with `BLACK`, then redraw just those zones.

4 Designing a zone layout

A zone layout divides the 128×64 display into regions, each responsible for one piece of information. This is the professional approach to embedded display design — it makes your screen predictable, maintainable, and flicker-free.

4.1 Planning the zones

Before writing code, sketch the layout. For an AHT20 environmental display, a natural three-zone layout works well:

Zone	Y range	Height	Content
Header	Y 0–15	16px	Title bar — label and status indicator
Temperature	Y 16–39	24px	Large temperature value, size 2 text
Humidity	Y 40–63	24px	Large humidity value, size 2 text

4.2 Zone layout implementation

Define zone constants at the top of your file — never use raw Y coordinates in drawing code:

```
// — Display zone constants —————
#define ZONE_HEADER_Y    0
#define ZONE_HEADER_H    16
#define ZONE_TEMP_Y      16
#define ZONE_TEMP_H       24
#define ZONE_HUMID_Y     40
#define ZONE_HUMID_H      24

// Draw the static elements once in setup()
void drawLayout() {
    oled.clearDisplay();

    // Header bar
    oled.fillRect(0, ZONE_HEADER_Y, 128, ZONE_HEADER_H, SSD1306_WHITE);
    oled.setTextColor(SSD1306_BLACK); // Inverted text on white bar
    oled.setTextSize(1);
    oled.setCursor(2, 4);
    oled.print("E4 Env Monitor");

    // Divider line between temp and humidity
    oled.drawFastHLine(0, 39, 128, SSD1306_WHITE);

    // Labels
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(1);
    oled.setCursor(0, ZONE_TEMP_Y + 1);
    oled.print("Temp");
    oled.setCursor(0, ZONE_HUMID_Y + 1);
    oled.print("Hum");

    oled.display();
}
```

```
}

// Update only the value zones when readings change
void updateValues(float tempC, float humidity) {
  // Erase temperature value area only
  oled.fillRect(36, ZONE_TEMP_Y, 92, ZONE_TEMP_H, SSD1306_BLACK);
  oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(2);
  oled.setCursor(36, ZONE_TEMP_Y + 4);
  oled.printf("%.1f C", tempC);

  // Erase humidity value area only
  oled.fillRect(36, ZONE_HUMID_Y, 92, ZONE_HUMID_H, SSD1306_BLACK);
  oled.setCursor(36, ZONE_HUMID_Y + 4);
  oled.printf("%.1f %%", humidity);

  oled.display();
}
```

`drawLayout()` runs once in `setup()` to draw the static chrome. `updateValues()` runs whenever a new reading arrives and touches only the value zones. The header and labels never flicker because they are never erased.

5 Complete project — AHT20 sensor display

This is the first complete sensor-to-screen project on the E4. It combines everything from T01 through T05 — millis() timing, build_flags, Button.h debounce, AHT20 I2C reading, and a zoned OLED display.

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_AHTX0.h>
#include "Button.h"

// — Display —————
#define OLED_WIDTH 128
#define OLED_HEIGHT 64
#define OLED_RESET -1
#define OLED_ADDR 0x3C

#define ZONE_HEADER_Y 0
#define ZONE_HEADER_H 16
#define ZONE_TEMP_Y 16
#define ZONE_TEMP_H 24
#define ZONE_HUMID_Y 40
#define ZONE_HUMID_H 24

Adafruit_SSD1306 oled(OLED_WIDTH, OLED_HEIGHT, &Wire, OLED_RESET);
Adafruit_AHTX0 aht;
Button nextBtn(BTN_NEXT);
Button entBtn(BTN_ENT);

// — State —————
unsigned long lastRead = 0;
unsigned long readInterval = 3000;
float lastTemp = 0.0;
float lastHumid = 0.0;
bool fastMode = false;

void drawLayout() {
  oled.clearDisplay();
  oled.fillRect(0, ZONE_HEADER_Y, 128, ZONE_HEADER_H, SSD1306_WHITE);
  oled.setTextColor(SSD1306_BLACK);
  oled.setTextSize(1);
  oled.setCursor(2, 4);
  oled.print("E4 Env Monitor");
  // Mode indicator top right
  oled.setCursor(86, 4);
  oled.print(fastMode ? "FAST" : "SLOW");
  oled.drawFastHLine(0, 39, 128, SSD1306_WHITE);
  oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(1);
  oled.setCursor(0, ZONE_TEMP_Y + 6);
  oled.print("Tmp");
  oled.setCursor(0, ZONE_HUMID_Y + 6);
  oled.print("Hum");
}
```

```
    oled.display();
}

void updateValues(float tempC, float humidity) {
    // Temperature zone
    oled.fillRect(24, ZONE_TEMP_Y, 104, ZONE_TEMP_H, SSD1306_BLACK);
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(2);
    oled.setCursor(24, ZONE_TEMP_Y + 4);
    oled.printf("%.1f C", tempC);

    // Humidity zone
    oled.fillRect(24, ZONE_HUMID_Y, 104, ZONE_HUMID_H, SSD1306_BLACK);
    oled.setCursor(24, ZONE_HUMID_Y + 4);
    oled.printf("%.1f %%", humidity);

    oled.display();
}

void readSensor() {
    sensors_event_t humidity, temp;
    aht.getEvent(&humidity, &temp);
    lastTemp = temp.temperature;
    lastHumid = humidity.relative_humidity;
    updateValues(lastTemp, lastHumid);
    Serial.printf("Temp: %.1f C Humidity: %.1f%%\n",
                  lastTemp, lastHumid);
}

void setup() {
    Serial.begin(115200);
    Serial.printf("T05 - AHT20 Display FW: %s\n", FW_VERSION);

    Wire.begin(I2C_SDA, I2C_SCL);

    pinMode(LED_GREEN, OUTPUT);
    pinMode(LED_RED, OUTPUT);
    pinMode(LED_BLUE, OUTPUT);
    digitalWrite(LED_GREEN, LOW);
    digitalWrite(LED_RED, LOW);
    digitalWrite(LED_BLUE, LOW);

    nextBtn.begin();
    entBtn.begin();

    bool ok = true;

    if (!oled.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
        Serial.println("ERROR: SSD1306 not found");
        ok = false;
    }

    if (!aht.begin(&Wire)) {
```

```

    Serial.println("ERROR: AHT20 not found");
    ok = false;
}

if (!ok) {
    digitalWrite(LED_RED, HIGH);
    while (true) { delay(100); }
}

digitalWrite(LED_GREEN, HIGH);
drawLayout();
readSensor(); // First reading immediately
Serial.println("Ready. NEXT=read  NEXT long=speed  ENT=refresh layout");
}

void loop() {
    unsigned long now = millis();
    Button::Event nextEv = nextBtn.read();
    Button::Event entEv = entBtn.read();

    // NEXT short: immediate read
    if (nextEv == Button::SHORT_PRESS) {
        lastRead = now;
        readSensor();
    }

    // NEXT long: toggle fast/slow
    if (nextEv == Button::LONG_PRESS) {
        fastMode = !fastMode;
        readInterval = fastMode ? 1000 : 3000;
        digitalWrite(LED_BLUE, fastMode ? HIGH : LOW);
        drawLayout(); // Redraw to update mode indicator
        Serial.printf("Mode: %s\n", fastMode ? "FAST" : "SLOW");
    }

    // ENT short: force full layout redraw
    if (entEv == Button::SHORT_PRESS) {
        drawLayout();
        updateValues(lastTemp, lastHumid);
        Serial.println("Layout refreshed.");
    }

    // Timed read
    if (now - lastRead >= readInterval) {
        lastRead = now;
        readSensor();
    }
}

```

5.1 What to observe

- The OLED shows a clean three-zone layout with the header bar, temperature, and humidity
- Values update every 3 seconds without any header or label flicker

- NEXT short press forces an immediate reading
- NEXT long press switches between SLOW and FAST modes — the header bar updates to show the current mode and BLUE LED confirms fast mode
- ENT short press forces a full layout redraw — useful if display corruption occurs
- GREEN LED confirms both OLED and AHT20 initialised successfully

Try this

Breathe on the AHT20 sensor and watch the humidity value climb in real time. Switch to FAST mode first for the most responsive display. This is your first complete embedded sensing and display project — it works exactly like a commercial IoT device.

6 Multi-page display driven by buttons

With only 128×64 pixels, you sometimes need more space than one screen can give. A multi-page approach lets you cycle through different views using NEXT — exactly how commercial embedded devices work.

6.1 Page architecture

The pattern is simple: an enum defines the pages, NEXT cycles through them, and a drawPage() function renders the correct content for the current page.

```
// Add to the project above — extend with additional pages

enum Page { PAGE_SENSOR, PAGE_STATS, PAGE_INFO, PAGE_COUNT };
Page currentPage = PAGE_SENSOR;

int readingCount = 0;
float minTemp = 999.0, maxTemp = -999.0;
float minHumid = 999.0, maxHumid = -999.0;

void drawPageSensor(float tempC, float humidity) {
    oled.clearDisplay();
    oled.fillRect(0, 0, 128, 14, SSD1306_WHITE);
    oled.setTextColor(SSD1306_BLACK);
    oled.setTextSize(1);
    oled.setCursor(2, 3);
    oled.print("Live Readings 1/3");
    oled.setTextColor(SSD1306_WHITE);
    oled.setTextSize(2);
    oled.setCursor(0, 18);
    oled.printf("%.1f C", tempC);
    oled.setCursor(0, 42);
    oled.printf("%.1f %%", humidity);
    oled.display();
}

void drawPageStats() {
    oled.clearDisplay();
    oled.fillRect(0, 0, 128, 14, SSD1306_WHITE);
    oled.setTextColor(SSD1306_BLACK);
    oled.setTextSize(1);
    oled.setCursor(2, 3);
    oled.print("Statistics 2/3");
    oled.setTextColor(SSD1306_WHITE);
    oled.setCursor(0, 18);
    oled.printf("Tmp Lo: %.1f Hi: %.1f", minTemp, maxTemp);
    oled.setCursor(0, 30);
    oled.printf("Hum Lo: %.1f Hi: %.1f", minHumid, maxHumid);
    oled.setCursor(0, 48);
    oled.printf("Readings: %d", readingCount);
    oled.display();
}

void drawPageInfo() {
```

```

    oled.clearDisplay();
    oled.fillRect(0, 0, 128, 14, SSD1306_WHITE);
    oled.setTextColor(SSD1306_BLACK);
    oled.setTextSize(1);
    oled.setCursor(2, 3);
    oled.print("Board Info 3/3");
    oled.setTextColor(SSD1306_WHITE);
    oled.setCursor(0, 18);
    oled.print("E4 Educator v2.0");
    oled.setCursor(0, 30);
    oled.printf("FW: %s", FW_VERSION);
    oled.setCursor(0, 42);
    oled.printf("Up: %lus", millis() / 1000);
    oled.display();
}

void refreshCurrentPage(float tempC, float humidity) {
    switch (currentPage) {
        case PAGE_SENSOR: drawPageSensor(tempC, humidity); break;
        case PAGE_STATS:  drawPageStats();                break;
        case PAGE_INFO:   drawPageInfo();                  break;
        default: break;
    }
}

// In loop(), replace the NEXT short press handler:
if (nextEv == Button::SHORT_PRESS) {
    currentPage = (Page)((currentPage + 1) % PAGE_COUNT);
    refreshCurrentPage(lastTemp, lastHumid);
}

// In readSensor(), after getting new values, update min/max:
readingCount++;
if (lastTemp < minTemp) minTemp = lastTemp;
if (lastTemp > maxTemp) maxTemp = lastTemp;
if (lastHumid < minHumid) minHumid = lastHumid;
if (lastHumid > maxHumid) maxHumid = lastHumid;
refreshCurrentPage(lastTemp, lastHumid);

```

★ Key point

The multi-page pattern scales to any number of pages without changing the button logic. Add a new enum value, add a new drawPageXxx() function, add it to the switch statement, and PAGE_COUNT automatically updates the cycling. This is the pattern used in every E4 Tier 2 and Tier 3 display project.

7 T05 summary

Concept	Key takeaway
SSD1306 OLED	128×64 monochrome, I2C address 0x3C, shares the existing I2C bus. OLED_RESET=-1 on the E4.
Display buffer	All drawing happens in memory. Nothing shows until oled.display() is called. Always build the full frame first then push once.

Zone layout	Define Y positions as constants. Draw static chrome once in setup(). Update only changed zones in loop(). No flicker.
Erasing zones	fillRect() with SSD1306_BLACK erases a rectangular area. Use this to clear a value zone before redrawing the new value.
Text sizes	Size 1 = 6×8px (21 chars/row), size 2 = 12×16px (10 chars/row), size 3 = 18×24px (7 chars/row).
Multi-page	enum for pages, NEXT cycles with % PAGE_COUNT, switch statement renders correct page. Scales to any number of pages.
Wire.begin()	Always call Wire.begin(I2C_SDA, I2C_SCL) before oled.begin(). Both AHT20 and OLED share the same Wire instance.

What's next

- T06 — One-wire and analog: the DS18B20 temperature sensor (one-wire protocol) and LDR light sensor (analog ADC1) complete your Tier 1 sensor toolkit.
- T07 — Audio output basics: piezo tones with LEDC, DAC sine wave audio, and PWM load control wrap up Tier 1 before the big step up to the TFT display in Tier 2.